

10. Write the python program to implement A* algorithm

Aim

To develop a Python program to implement the A* Search Algorithm for finding the shortest path between two nodes in a graph using heuristic values.

Algorithm

1. Start the program.
 2. Read the graph, heuristic values, start node, and goal node from the user.
 3. Initialize the Open List with the start node and an empty Closed List.
 4. Assign the cost values:
 - $g(n)$ = Cost from the start node.
 - $h(n)$ = Heuristic value.
 - $f(n) = g(n) + h(n)$.
 5. Repeat until the Open List becomes empty:
 - Select the node with the minimum $f(n)$ value.
 - If the selected node is the goal node, display the shortest path and stop.
 - Move the selected node to the Closed List.
 - Generate all neighboring nodes.
 - Compute $g(n)$, $h(n)$, and $f(n)$ for each neighbor.
 - Update the Open List if a better path is found.
 6. If no path exists, display "Path not found."
 7. Stop.
-

Program

```
import heapq

# Read graph
graph = {}

n = int(input("Enter the number of nodes: "))

print("\nEnter node details:")
```

```

for _ in range(n):
    node = input("Node: ")
    m = int(input(f"Enter number of neighbors of {node}: "))
    graph[node] = []

    for i in range(m):
        neighbor = input("Neighbor: ")
        cost = int(input("Cost: "))
        graph[node].append((neighbor, cost))

# Read heuristic values
heuristic = {}

print("\nEnter Heuristic Values:")
for node in graph:
    heuristic[node] = int(input(f"h({node}) = "))

# Read start and goal
start = input("\nEnter Start Node: ")
goal = input("Enter Goal Node: ")

# A* Algorithm
pq = []
heapq.heappush(pq, (heuristic[start], 0, start, [start]))

visited = {}

while pq:
    f, g, current, path = heapq.heappop(pq)

    if current == goal:
        print("\nShortest Path:", " -> ".join(path))
        print("Total Cost:", g)
        break

```

```
    if current in visited and visited[current] <= g:
        continue

    visited[current] = g

    for neighbor, cost in graph[current]:
        new_g = g + cost
        new_f = new_g + heuristic[neighbor]

        heapq.heappush(
            pq,
            (new_f, new_g, neighbor, path + [neighbor])
        )
else:
    print("Path not found.")
```

Input

Enter the number of nodes: 4

Enter node details:

Node: A

Enter number of neighbors of A: 2

Neighbor: B

Cost: 1

Neighbor: C

Cost: 4

Node: B

Enter number of neighbors of B: 2

Neighbor: C

Cost: 2

Neighbor: D

Cost: 5

Node: C

Enter number of neighbors of C: 1

Neighbor: D

Cost: 1

Node: D

Enter number of neighbors of D: 0

Enter Heuristic Values:

$h(A) = 7$

$h(B) = 6$

$h(C) = 2$

$h(D) = 0$

Enter Start Node: A

Enter Goal Node: D

Output

```
Shortest Path: A -> C -> D
Total Cost: 5
>>> |
```

Result

The Python program was successfully implemented to perform the A* Search Algorithm. The program accepts the graph, heuristic values, start node, and goal node from the user, computes the shortest path using the evaluation function $f(n) = g(n) + h(n)$, and displays the optimal path along with the minimum total cost.